

ОПИСАНИЕ ПРОЕКТА

Тема проекта:

Онлайн-платформа для бытовых мастеров и клиентов, где клиенты размещают заявки на услуги или обращаются к мастерам напрямую, мастера ведут профили и принимают заказы, а администрация управляет отзывами и частью сущностей системы.

Актуальность:

Актуальность разработки данной системы обусловлена устойчивым ростом потребности в цифровых сервисах, обеспечивающих оперативный и удобный поиск исполнителей бытовых услуг. Традиционные способы поиска мастеров не всегда позволяют обеспечить достаточный уровень доступности, прозрачности и надёжности взаимодействия между сторонами. Создание специализированной онлайн-платформы позволяет структурировать процесс оказания бытовых услуг, упростить взаимодействие между клиентами и мастерами, а также повысить качество обслуживания за счёт использования механизмов учёта заявок, откликов, заказов и отзывов.

Назначение:

Проектируемая система предназначена для автоматизации процессов взаимодействия между клиентами, нуждающимися в бытовых услугах, и мастерами, предоставляющими соответствующие услуги. Система обеспечивает возможность размещения клиентских заявок, обработки откликов мастеров, оформления заказов, а также хранения и отображения отзывов по результатам выполненных работ.

Целевая аудитория:

Целевую аудиторию системы составляют три основные группы пользователей: клиенты, заинтересованные в поиске исполнителей для получения бытовых услуг; мастера, предоставляющие услуги по различным направлениям; администраторы системы, осуществляющие управление отдельными сущностями платформы, а также контроль её функционирования.

Стек

Основной ЯП: C#

Версия платформы: .NET 10.0

Backend-Фреймворк: ASP.NET Core Web API

СУБД: PostgreSQL

ORM: EF Core

UI: Web UI

Состав команды:

Авдиенко Данила, Говоров Павел

ПРОЕКТИРОВАНИЕ БД

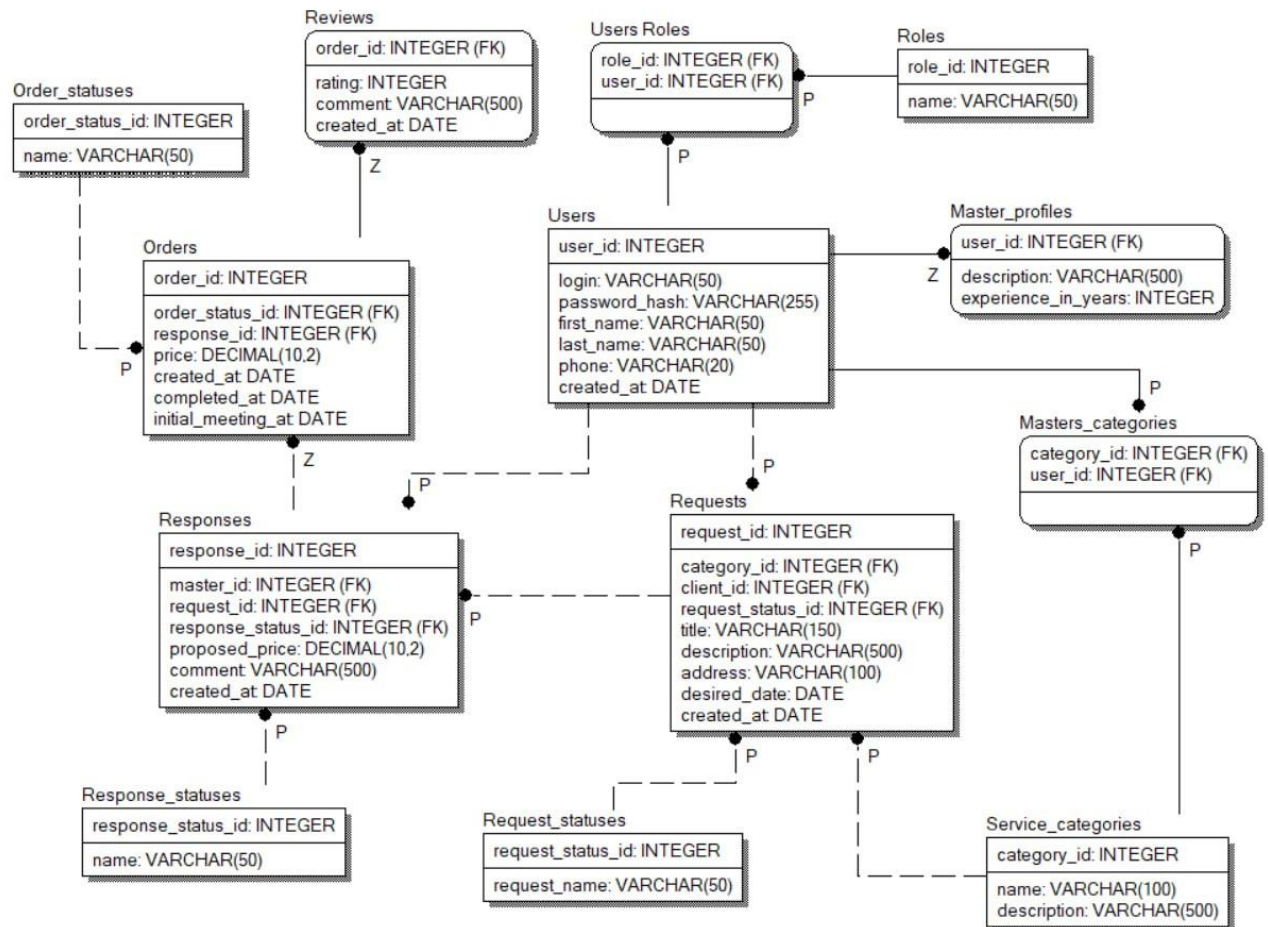
Таблицы

1. users — учетные записи всех пользователей системы
2. roles — типы пользователей в системе (клиент, мастер, администратор)
3. user_roles — назначение ролей конкретным пользователям
4. service_categories — категории бытовых услуг
5. master_profiles — профессиональные профили мастеров
6. master_categories — специализации мастеров по категориям услуг
- (request-response-order)**
7. requests — заявки клиентов на выполнение работ
8. request_statuses — возможные статусы заявок
9. responses — отклики мастеров на клиентские заявки (список откликов каждого мастера)
10. response_statuses — возможные статусы откликов мастеров
11. orders — согласованные заказы между клиентом и мастером
12. order_statuses — возможные статусы заказов

13. reviews — отзывы клиентов по завершённым услугам

разделение **request-response-order** нужно, чтобы база хранила не “всё подряд про услугу”, а разные стадии процесса по отдельности: сначала клиент создал заявку, потом мастер откликнулся, потом стороны оформили заказ.

IDEF1X



ФУНКЦИОНАЛЬНЫЕ ТРЕБОВАНИЯ

No-login

- Система должна предоставлять доступ к главной странице платформы
- Система должна предоставлять возможность ознакомиться с базовой информацией по платформе (фиксированный список мастеров, примеры заявок, статистика, например рассчитываемое количество записей в открытых заявках/ числе мастеров)
- Система должна предоставлять возможность неавторизованному пользователю перейти к регистрации или входу в аккаунт

Регистрация и авторизация

- Система должна предоставлять пользователю возможность зарегистрировать аккаунт с выбранной ролью
- Система всегда должна запрашивать при регистрации логин, пароль, имя, фамилию, номер телефона
- Система должна проверять уникальность логина при регистрации
- Система должна предоставлять пользователю выбор одной роли при регистрации (мастер/клиент)
- Система должна сохранять данные нового пользователя в базе данных в таблицу users, создавать запись о роли пользователя в user_roles.
- Система должна предоставлять возможность входа в систему по логину и паролю без указания роли пользователя при входе
- Система должна проверять корректность введенных данных при входе

Client-role

Профиль

- Система должна предоставлять пользователю возможность просматривать данные своего профиля
- Система должна предоставлять возможность пользователю редактировать имя, фамилию, номер телефона и пароль от аккаунта

ДОПОЛНИТЕЛЬНО

- Система должна предоставлять рассчитываемую информацию (о времени, прошедшем с момента регистрации на сайте; количество завершенных заказов)

Просмотр мастеров

Ввиду того, что в нашей бд не поддерживается логика direct-request от клиента к конкретному мастеру, и разработка всего, прилагающегося к этому, функционала займет много времени, смысла делать вкладку просмотра мастеров, как закрытую витрину без direct-откликов, нет, хоть и выглядит эта часть как обязательная для такого продукта.

Заявки пользователя

- Система должна предоставлять пользователю возможность создавать новую заявку на получение бытовой услуги
- Система должна запрашивать при создании заявки категорию услуги, заголовок, описание, адрес и желаемую дату выполнения
- Система должна сохранять созданную заявку в базе данных
- Система должна предоставлять пользователю возможность отменить свою заявку
- Система должна предоставлять пользователю возможность просматривать список своих заявок

Мы не будем предоставлять возможность редактирования заявки потому что это тянет ряд вопросов за собой, например: что делать с существующими откликами, если заявка была изменена? И т.д.

Отклики на заявки

- Система должна предоставлять пользователю возможность просматривать отклики мастеров на свои заявки
- Система должна отображать в отклике информацию о мастере, предложенную цену и комментарий
- Система должна предоставлять пользователю возможность выбрать один из откликов по своей заявке

Триггер - Система должна инициировать создание заказа на основе принятого отклика

Триггер - При выборе одного отклика на заявку, все остальные отклики должны получить статус rejected

Завершенные заказы

- Система должна предоставлять пользователю возможность просматривать историю завершённых заказов
- Система должна отображать в карточке завершённого заказа информацию о нем (из orders, responses, requests)
- Система должна отображать статус и дату завершения заказа.
- Система должна предоставлять пользователю возможность перейти к оставлению отзыва по завершённому заказу.

Отзывы

- Система должна предоставлять пользователю возможность оставить отзыв по завершённому заказу
- Система должна запрашивать при создании отзыва оценку и опциональный текстовый комментарий
- Система должна сохранять отзыв в базе данных

Отображение отзывов

Система должна отображать отзыв как отзыв о мастере по выполненной услуге. Для отображения отзыва система должна определять мастера и категорию услуги через связанные сущности заказа, отклика и заявки.

Master-role

Профиль мастера

Система должна предоставлять мастеру возможность просматривать данные своего аккаунта

Система должна предоставлять мастеру возможность редактировать имя, фамилию, номер телефона и пароль от аккаунта

Система должна предоставлять мастеру возможность просматривать данные своего профессионального профиля

Система должна предоставлять мастеру возможность редактировать данные своего профессионального профиля

Система должна предоставлять мастеру возможность просматривать список своих специализаций

Система должна предоставлять мастеру возможность добавлять и удалять свои специализации

Просмотр доступных заявок

Система должна предоставлять мастеру возможность просматривать список открытых заявок клиентов, которые соответствуют его специализациям

Система должна отображать по каждой заявке основную информацию: категорию услуги, заголовок, описание, адрес, желаемую дату выполнения, дату создания заявки и её статус

Система должна скрывать или помечать заявки, на которые мастер уже откликнулся

Система должна предоставлять мастеру возможность фильтровать и сортировать заявки

Отклики мастера на заявки

Система должна предоставлять мастеру возможность оставить отклик на заявку клиента

Система должна запрашивать при создании отклика предлагаемую цену и опциональный комментарий

Система должна запрещать мастеру откликаться на заявку, которая находится в неактуальном статусе, например отменена, закрыта или уже переведена в заказ

Система должна предоставлять мастеру возможность просматривать список своих откликов

Система должна предоставлять мастеру возможность отозвать свой отклик, пока клиент не выбрал исполнителя

Триггер

Система должна автоматически закрывать заявку после создания заказа

Работа с заказами мастера

Система должна предоставлять мастеру возможность просматривать список заказов, сформированных на основе его принятых откликов

Система должна предоставлять мастеру возможность просматривать активные, завершённые, отменённые заказы

Триггер

Система должна автоматически устанавливать дату завершения заказа при переводе его в статус завершённого

Отзывы о мастере

Система должна предоставлять мастеру возможность просматривать отзывы, оставленные клиентами по завершённым заказам

Триггер

Система должна запрещать создание второго отзыва для одного и того же заказа через проверку в бд

Триггер

Система должна запрещать создание отзыва для заказа, который не находится в статусе completed

Usecases:

1. Регистрация и вход пользователя

- Пользователь открывает страницу регистрации, вводит личные данные и выбирает роль: клиент или мастер.
- Система проверяет данные, создаёт пользователя и назначает ему выбранную роль.
- Пользователь входит в систему по логину и паролю.
- Система определяет роль пользователя и открывает доступ к соответствующему личному кабинету.

2. Создание заявки клиентом

- Клиент открывает раздел заявок.
- Клиент заполняет форму: категория услуги, заголовок, описание, адрес и желаемая дата выполнения.
- Система проверяет данные и сохраняет заявку.
- Заявка появляется в списке заявок клиента и становится доступна мастерам.

3. Создание отклика мастером

- Мастер открывает список доступных заявок.
- Система отображает заявки, подходящие под специализации мастера.
- Мастер выбирает заявку и указывает предлагаемую цену и комментарий.
- Система сохраняет отклик и показывает его клиенту по соответствующей заявке.

4. Выбор отклика и создание заказа

- Клиент открывает свою заявку и просматривает отклики мастеров.
- Клиент выбирает подходящий отклик.
- Система принимает выбранный отклик и отклоняет остальные отклики по этой заявке.
- Система создаёт заказ на основе выбранного отклика.

5. Выполнение заказа

- Клиент и мастер видят созданный заказ в своих личных кабинетах.
- Система отображает данные заказа: заявку, мастера, клиента, цену и статус.
- После выполнения услуги заказ переводится в завершённый статус.
- Завершённый заказ сохраняется в истории заказов.

6. Оставление отзыва

- Клиент открывает завершённый заказ.
- Клиент указывает оценку и при необходимости комментарий.
- Система проверяет, что по заказу ещё нет отзыва.
- Система сохраняет отзыв и отображает его в профиле мастера.

Проектирование API

За основу API взят REST API.

API будет строиться вокруг ресурсов нашего проекта.

Ресурсы это объекты или коллекции объектов, с которыми клиент работает через API. Не все таблицы БД являются ресурсами, так как не со всеми из них фронтенд работает напрямую.

Не все таблицы БД – ресурсы И не все ресурсы – таблицы БД

Если frontend получает их отдельным запросом — это API-ресурс

Если они приходят внутри другого ответа — это не отдельный API-ресурс

Список ресурсов:

- Auth – вход/регистрация
- Users – получение и изменение данных профиля пользователя
- Masters – получение и изменение данных мастера
- Requests - создание/деактивация/просмотр
- Responses – просмотр откликов на заявку пользователя/выбор отклика; создание отклика на заявку/просмотр списка откликов мастера/отмена отклика
- Orders – просмотр одного и списка как клиента, так и мастера
- Reviews – публикация отзывов на заверченный заказ и просмотр отзывов мастера
- ServiceCategories – получение списка категорий услуг

API conventions:

- Все эндпоинты начинаются с /api
- Имена ресурсов пишутся во множественном числе
- В названиях, где больше одного слова, используем kebab-case (Пример: /api/service-categories)

HTTP methods

- GET – получение данных
- POST – создание сущности или выполнение бизнес действия
- PATCH – изменение части существующих данных
- DELETE – удаление/отмена данных

DTO conventions:

/ DTO – объект для передаче через API (Data Transfer Object). Благодаря им мы можем выбирать, какие данные отдаем, какие принимаем.

Request DTO – то, что отправляет клиент в бэкенд

Response DTO – то, что отправляет бэкенд клиенту /

- DTO, принимающие данные от клиента, заканчиваются на Request
- DTO, отдающие данные от бэкенда (к клиенту), заканчиваются на Response
- Request DTO называются по действию
 - RegisterRequest
 - LoginRequest
 - CreateReviewRequest
 - И так далее
- Response DTO называются по возвращаемому ресурсу
 - AuthResponse
 - UserProfileResponse
 - OrderResponse
 - ReviewResponse

Проектирование API + DTO по ресурсам

Auth

Api contracts

- POST /api/auth/register – регистрация пользователя (в тч роль)
- POST /api/auth/login – вход в систему

Проверять уникальность login при регистрации

Dto

RegisterRequest:

- login
- password
- firstName
- lastName
- phone
- role

LoginRequest:

- login
- password

AuthResponse:

- userId
- login
- role
- token

(AuthResponse используется и при регистрации, и при логине)

Users

Api contracts

- GET /api/users/me - получение данных текущего пользователя
- PATCH /api/users/me – изменение данных профиля пользователя
- PATCH /api/users/me/password – изменение пароля пользователя
(не возвращает ничего, просто подтверждает успешное изменение пароля)

DTO

UserProfileResponse

- userId
- login
- firstName
- lastName
- phone
- role
- createdAt
- daysSinceRegistration (рассчитывается)
- completedOrdersCount (рассчитывается)

UpdateUserProfileRequest

- firstName
 - lastName
 - phone
- (все поля опциональны)

ChangePasswordRequest

- currentPassword
- newPassword

Masters

API contracts

- GET /api/masters/me – получение данных текущего пользователя + уникальные данные для роли мастер
- PATCH /api/masters/me – изменение описания и опыта работы
- GET /api/masters/me/categories – получение списка категорий мастера (возвращается в формате списка из составных объектов [(categoryId, name), (...)])
- PUT /api/masters/me/categories – изменение списка собственных специализаций мастера (заменяет весь список на новый)

DTO

MasterProfileResponse:

- userId
- login
- firstName
- lastName
- phone
- description
- experienceYears
- categories: List<ServiceCategoriesResponse>

UpdateMasterProfileRequest:

- description
- experienceYears

UpdateMasterCategoriesRequest:

- categoryIds

ServiceCategories

API contracts

- GET /api/service-categories – получение существующего списка категорий, который может быть использован пользователями с любой ролью. Возвращается в формате списка из составных объектов [(categoryId, name), (...)]

DTO

ServiceCategoryResponse (в том числе и для получения категорий конкретного мастера):

- categoryId
- name

Orders

API contracts

GET /api/orders/{orderId} – получение одного заказа по id

GET /api/users/me/orders – получение списка заказов текущего пользователя-клиента (вернет List<UserOrderListItemResponse>)

GET /api/masters/me/orders – получение списка заказов текущего мастера (вернет MasterOrderListItemResponse)

PATCH /api/orders/{orderId}/complete — завершение заказа клиентом (берется айди клиента из заказа и сравнивается с айди в токене и там же проверяется роль пользователя, чтобы она соответствовала клиенту)

PATCH /api/orders/{orderId}/cancelled – отмена заказа в одностороннем порядке. Может быть сделана как `

DTO

OrderResponse:

- orderId
- status
- price
- initialMeetingAt
- createdAt
- completedAt (nullable)

- requestId
- requestTitle
- requestDescription
- requestAddress
- desiredDate

- categoryId
- categoryName
- clientId
- clientFirstName
- clientLastName
- clientPhone
- masterId
- masterFirstName
- masterLastName
- masterPhone

UserOrderListItemResponse:

- orderId
- status
- price
- initialMeetingAt
- createdAt
- completedAt
- requestId
- requestTitle
- categoryName
- masterId
- masterFirstName
- masterLastName

MasterOrderListItemResponse:

- orderId
- status
- price
- initialMeetingAt
- createdAt
- completedAt
- requestId
- requestTitle
- categoryName
- clientId
- clientFirstName
- clientLastName
- clientPhone

Reviews

API contracts

POST /api/orders/{orderId}/reviews – создание отзыва по завершённомu заказу

GET /api/masters/{masterId}/reviews – получение списка отзывов, оставленных по заказам конкретного мастера (вернет List<MasterReviewListItemResponse>)

Один заказ может иметь только один отзыв.

DTO

CreateReviewRequest:

- rating
- comment

MasterReviewListItemResponse:

- reviewId
- orderId
- rating
- comment
- createdAt

- clientFirstName
- clientLastName

- requestTitle
- categoryName

Requests

API contracts

POST /api/requests – создание новой заявки клиентом

GET /api/requests/{requestId} – получение одной заявки по id

PATCH /api/requests/{requestId}/cancel — отмена заявки клиентом

GET /api/users/me/requests – получение списка заявок текущего клиента

GET /api/requests – получение списка открытых заявок для мастера

Заявку создаёт только пользователь с ролью client.

Список `/api/users/me/requests` нужен клиенту для просмотра своих заявок.
Список `/api/requests` нужен мастеру для просмотра доступных открытых заявок.

DTO

CreateRequestRequest:

- `categoryId`
- `title`
- `description`
- `address`
- `desiredDate`

RequestResponse:

- `requestId`
- `clientId`
- `clientFirstName`
- `clientLastName`
- `categoryId`
- `categoryName`
- `status`
- `title`
- `description`
- `address`
- `desiredDate`
- `createdAt`

UserRequestListItemResponse:

- `requestId`
- `categoryId`
- `categoryName`
- `status`
- `title`
- `desiredDate`
- `createdAt`

AvailableRequestListItemResponse:

- `requestId`
- `clientId`
- `clientFirstName`
- `clientLastName`
- `categoryId`
- `categoryName`
- `status`
- `title`
- `description`

- address
- desiredDate
- createdAt

Responses

API contracts

POST /api/requests/{requestId}/responses – создание отклика мастером на заявку клиента

GET /api/requests/{requestId}/responses – получение списка откликов по конкретной заявке клиента

GET /api/masters/me/responses – получение списка откликов текущего мастера

POST /api/responses/{responseld}/accept — принятие отклика клиентом

POST /api/responses/{responseld}/cancel — отмена отклика мастером

При создании отклика система должна проверять, что заявка находится в активном статусе, а мастер еще не оставлял отклик на эту заявку.

DTO

CreateResponseRequest:

- proposedPrice
- comment

ResponseForRequestListItemResponse:

- responseld
- requestId
- status
- proposedPrice
- comment
- createdAt

- masterId
- masterFirstName
- masterLastName
- masterPhone
- masterDescription
- masterExperienceYears

MasterResponseListItemResponse:

- responsId
- requestId
- requestTitle
- categoryName
- status
- proposedPrice
- comment
- createdAt

Коды ответов:

Для успешных операций API возвращает стандартные HTTP-коды:

200 OK – запрос успешно выполнен, сервер возвращает данные.

201 Created – сущность успешно создана. Используется, например, при регистрации пользователя, создании заявки, отклика или отзыва.

204 No Content – операция успешно выполнена, но тело ответа не возвращается. Используется, например, при изменении пароля, отмене заявки или отзыве отклика.

Коды ошибок:

400 Bad Request – некорректный запрос.

Возвращается, если клиент передал неверные или неполные данные.

Например:

- не заполнено обязательное поле;
- цена отклика меньше или равна нулю;
- дата заявки указана в прошлом;
- оценка отзыва находится вне допустимого диапазона.

401 Unauthorized – пользователь не авторизован.

Возвращается, если пользователь пытается выполнить действие без входа в систему или без действительного токена.

403 Forbidden – недостаточно прав для выполнения действия.

Возвращается, если пользователь авторизован, но его роль не позволяет выполнить действие.

Например:

- клиент пытается создать отклик;
- мастер пытается оставить отзыв;
- пользователь пытается изменить чужой профиль.

404 Not Found – ресурс не найден.

Возвращается, если запрашиваемая сущность не существует или недоступна пользователю.

Например:

- заявка с указанным id не найдена;
- отклик с указанным id не найден;
- заказ с указанным id не найден;
- мастер с указанным id не найден.

409 Conflict – конфликт текущего состояния данных.

Возвращается, если действие невозможно выполнить из-за бизнес-правил или текущего состояния сущности.

Например:

- мастер уже оставил отклик на эту заявку;
- заявка уже отменена или закрыта;
- отклик уже принят или отклонён;
- заказ уже имеет отзыв;
- логин уже занят при регистрации.

500 Internal Server Error – внутренняя ошибка сервера.

Возвращается при непредвиденной ошибке на стороне сервера.
Клиенту не передаются технические детали ошибки.